Background: Duke Kunshan University (DKU) is an interdisciplinary institution that grants dual undergraduate degrees, an MOE Chinese degree and a degree from Duke University in Durham, United States. The principal structure of DKU majors is robustly interdisciplinary. No student confines their study to a single discipline (for example, biology or economics). Instead, all students engage in broad inquiry related to a subject or question (for example, political economy or global health) and take a wide variety of courses related to that area (for example, in public policy, history, ethics, or economics). As a result, our graduates are prepared to engage in a wide variety of inquiries using multiple methodologies to address complex issues that require interdisciplinary approaches.

This has implications for our vision and expectations of undergraduate theses and design projects, which reflect this broad interdisciplinary training. At DKU, every student completes a two-year project known as signature work which consists of multiple interconnected parts including thematic courses, experiential learning, capstones, and a final product. It seeks to integrate students' interdisciplinary educational experience and culminates in the creation of a product in a scholarly, creative, or applied nature in lieu of an undergraduate thesis or design required by JED. Because DKU encourages students to cultivate their independence and creativity as one of its institutional student learning outcomes, the student-led signature work projects often reflect students' own particular interdisciplinary interests and training. In addition, signature work has an intensive emphasis on problem-solving and skill-development which is much needed for any interdisciplinary inquiry; thus, students' final products are evidence of transferrable skills that students have acquired and demonstrated through the 2-year program, rather than content knowledge narrowly defined by disciplinary training.

In sum, while the Chinese major declared with any given student might be construed narrowly, the experience of our students is much broader—and intentionally so. This is a distinctive feature of our curriculum, and this distinctiveness results in broadly interdisciplinary submissions from our graduates' submitting theses or design projects. We have designed this to prepare our students for a wide variety of graduate programs in China and the West, where interdisciplinary training is a competitive advantage.

# GROUNDLOOP: A SELF-CHECKING AGENTIC RAG SYSTEM WITH EVALUATION-GUIDED RECOVERY

by

Sean Allen Siegfreid R. Bugarin

Signature Work Product, in partial fulfillment of the
Duke Kunshan University Undergraduate Degree Program

*November 2, 2026*

Signature Work Program
Duke Kunshan University

**APPROVALS**

---

Mentor: Bing Luo, Division of Natural and Applied Sciences

# CONTENTS


Abstract ii

Acknowledgements iii

List of Figures iv

List of Tables v

1 Introduction 1

2 The Product: GroundLoop 4

3 Evaluation Method 9

4 Results 13

5 Discussion 17

6 Conclusions 19

References 21

Signature Work Narrative 24

A Additional Material 29

# ABSTRACT

GroundLoop is a functional implementation of a question answering system that operates on its own documents and checks its own answers. It asks questions, retrieves passages and answers, scores the answer with a language model judge and deterministic tests, diagnoses why it failed, recovers by rewriting the question, widening retrieval, regenerating the answer, or abstaining, verifies the result, and reports everything through a web console. Runs are traced forever. The paper then explores how much trust we can place in this self-check. GroundLoop was compared to a retrieve-once baseline with 1, 2, and 4 passages and a plain 8 passage baseline on three small collections (88 questions in total), three times. It improved correct answers from 0.81 to 0.97 on one collection and to 1.00 on the other two, with just one actual regression in 27 runs of the loop. The 8-passage baseline matched it for less than half the cost. The diagnosis matched a gold class offline most of the time, and the verdict matched gold on 59 out of 73 times when an answer could be recovered. The loop reported a repair after changing the source an answer named.

**Keywords:** GroundLoop; retrieval-augmented generation; agentic RAG; runtime evaluation; recovery verification.

# ACKNOWLEDGEMENTS

I thank Bing Luo for mentoring this Signature Work, and Mingxi, a doctoral student in his group, who guided my research on ChatDKU. That work on evaluating a campus question-answering system is what led to the question this project studies. Bing Luo and Mingxi also provided the language-model API keys used to run the experiments. I also thank Paul Weng, whose summer research first introduced me to evaluating AI systems.

# LIST OF FIGURES

2.1 One query through the evaluation-guided loop. The dashed arrow is a retry while the recovery budget remains. . . . . 5
2.2 The Test Console after an automatic recovery. The loop strip shows the stage results, and the right panel lists the retrieved passages with their scores. . . . . 6
2.3 A recovered run in the Reliability view, with the runtime verdict and the initial and recovered answers side by side. . . . . 7
2.4 The Reliability summary for the Duke Kunshan system. . . . . 7

A.1 The Overview page for the Duke Kunshan system. . . . . 30
A.2 The Configuration page. Built-in systems are read-only; a new system is fully editable. . . . . 31
A.3 The Runs view of the Reliability page, with one run opened. . . . . 31
A.4 The Data page, listing a system's sources with one open. . . . . 32
A.5 The Integrations page for the external evaluation API. . . . . 32

# LIST OF TABLES


3.1 Study collections and gold question counts. . . . . 10

3.2 Default recovery policy. . . . . 11

4.1 Contains-answer rate on answerable questions, and abstention rate on unanswerable questions. Mean of three repeats. Harbor has 23 answerable and 9 unanswerable questions, Duke Kunshan 30 and 10, Kestrel 12 and 4. . . . . 13

4.2 Questions fixed and broken by the loop compared to the baseline (same depth). 14

4.3 Loop diagnosis vs. the gold failure class. $k = 1$, nine runs. . . . . 15

4.4 Chat calls per run, estimated cost per run (in US dollars), and median latency per question (in seconds). Mean across three repeats. . . . . 15

4.5 Runtime verdict against the gold rank transition, all loop runs. . . . . 16

A.1 Earlier $k = 4$ runs, stored and rescored . . . . . 30

Chapter 1

# INTRODUCTION

## 1.1 Motivation

This project grew out of my work as a researcher on ChatDKU, a campus question-answering assistant at Duke Kunshan University. Most of that role was evaluation. A score told us how often the assistant was right. It did not tell us what had gone wrong or which change would help: a low score could come from retrieval, from the documents, or from the model's answer. I also started an automated evaluation and did not finish it. That gap between scoring a system and knowing how to repair it is the starting point for this work.

## 1.2 Problem and Research Question

Retrieval augmented generation (RAG) systems comprise a passage retrieval component and use a language model to answer from the passages [1]. Errors are silent: wrong passage, missing fact, fluent answer with unsupported claims. An agentic RAG system does not settle for the first pass answer. It acts, evaluates the answer, diagnoses why it failed, then recovers by rewriting the answer, retrieving more passages, or abstaining from answering, and verifies whether the correction was effective:

$$\text{act} \rightarrow \text{evaluate} \rightarrow \text{diagnose} \rightarrow \text{recover} \rightarrow \text{verify} \rightarrow \text{accept, retry, or abstain.}$$

The "accept, retry, or abstain" step uses the same automatic answer evaluator that triggered the failure in the first place. If the evaluator is wrong, the system will accept a bad answer or claim a correction when there was none.

> Can the runtime evaluation of an agentic RAG system diagnose causes of answer failure and select an appropriate correction strategy given that failure cause, and judge whether the correction improved the answer?

The question has four components: (1) the comparison of the evaluation-guided loop to a retrieve-once baseline, for various depths of retrieval and for retrieving more passages; (2) the agreement of the loop's diagnosis to an offline gold standard failure class; (3) the agreement of the loop's verification verdict to an offline gold standard check; (4) the agreement of the judge's verdict to a human reader. Single-hop retrieval is used; this is a limitation of the Scope, not of the question.

## 1.3 Background and Prior Work

RAG has a retriever and generator, with the generator conditioned on the retrieved results [1]. Recent surveys document a shift from a fixed pipeline RAG to modular or agentic RAG, where an agent controls when and what to retrieve [2], [3]. Adaptive approaches take a runtime signal to determine when to retrieve: FLARE uses low token confidence, [4], Self-RAG uses reflection tokens learned during training [5], and CRAG scores documents and refines or replaces them [6], while Adaptive-RAG routes based on question complexity [7]. They report accuracy on the task at hand. They do not assess whether the system was correct in claiming that the action was helpful.

Offline diagnostic tools: RAGAs, ARES, and RAGChecker provide faithfulness, relevance, and module-level error scores [8], [9], [10]. Barnett et al. enumerate seven ways RAG can fail and point out that validation can only occur at runtime [11]. Joren et al. distinguish between insufficient and unused context, and use sufficiency to trigger abstention [12]. Language model judges can match humans on preference rating, but do so with position and length biases [13], [14]. Self-correction works well when feedback is reliable and often does not otherwise [15],

[16], [17]. Agent failure taxonomies address diagnosis and correction separately [18], [19]. Abstention is a common mechanism for controlling hallucination [20].

These systems either take an action at runtime and report accuracy on the end task or perform a post-run diagnosis. This project has two components. First, the product: GroundLoop is an app which runs the entire loop on a set of the user's documents and diagnoses, action, and verdict each answer, visible to the user as to why the system acted. Second, the measurement of that self-assessment: the diagnoses and verdicts of the loop are scored against gold that the running system does not have access to, and the judge against a human check. This study tests the product, not that proposes a new retriever.

## 1.4 Scope and Limits

The work is limited in four deliberate ways.

- **Single-hop retrieval.** Each question retrieves once, then at most two recoveries that may retrieve again. Multi-hop chaining is out of scope.
- **Small, controlled collections.** Two fictional sets and one set of public Duke Kunshan facts, written so each gold answer can be checked by hand.
- **Runtime signals never read the answer key.** The judge, diagnosis, policy, and verifier see only the question, the passages, and the answer. Gold is used only offline.
- **Software is the primary deliverable.** Experimental claims are limited to the recorded runs on these collections.

## Chapter 2

# THE PRODUCT: GROUNDLOOP

The primary deliverable of this Signature Work is a working application, GroundLoop. It lets a person build a question-answering system over their own documents, watch every answer go through the evaluation-guided loop, and see why each recovery happened and whether it helped. This chapter describes what the software does, how it is built, and the decisions behind it; Appendix A.1 gives where to find and run it. Chapters 3 and 4 then use the same software as an instrument to measure the loop.

## 2.1 Design Goals

Three things came out of the ChatDKU work that inspired this project:

- **Explain, don't score.** The score only told how often the assistant was correct, nothing about what could go wrong. Each answer should have a log about what was retrieved, what the judge thought, which failure was diagnosed, what was done to fix it, and how it changed.
- **Honest judge.** Since the loop judges itself, whatever it says cannot be taken as the truth. The runtime cannot look at the answer key, and the interface must mark the verdict as a comparison between judge flags, not a comparison to correctness.
- **Use by someone who didn't build it.** No code or JSON should be necessary to build, populate, and test a system. This was a guiding principle for the design, and the claim was

not validated by actual users.

## 2.2 Architecture

GroundLoop has three layers, and dependencies point one way. Figure 2.1 shows the loop the core runs.

- **Core library** (Python, package `swrag`, about 15,000 lines in 83 modules). It covers ingestion of text and PDF files, retrieval, generation, evaluation, diagnosis, recovery, verification, tracing, and the benchmark runner. It runs from the command line and from tests with no server.
- **HTTP adapter** (FastAPI, 28 routes). It exposes runs, recovery, traces, systems, documents, configuration, API keys, and an external evaluation API. The core never imports the adapter.
- **Web console** (plain HTML, CSS, and JavaScript, no build step). It renders only what the adapter returns.

The *trace* ties the layers together. Every query writes one JSON file that is never overwritten: the retrieved passages and their scores, the judge flags and rationales, the diagnosis, each recovery attempt with its parameters, latency, and token use, and the verification verdict. A recovery applied after an operator confirms it is a child trace linked to its parent. The console, the reliability statistics, and the benchmark tables are all computed from these files. There is no second record that could drift from what actually ran.

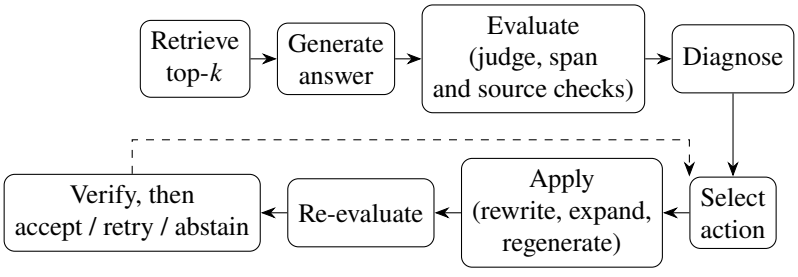



Figure 2.1: One query through the evaluation-guided loop. The dashed arrow is a retry while the recovery budget remains.

## 2.3 Using GroundLoop

An operator builds the system by uploading its documents and specifying its instructions, chat model, retrieval depth, and recovery policy on the Configuration page. The Test Console (Figure 2.2) is used for asking questions. There are four run modes:

- **Retrieve only** requires no API key and just shows the passages that would be retrieved
- **Baseline** produces an answer once
- **Confirm recovery** pauses after the diagnosis step and requires confirmation before proceeding
- **Auto recovery** runs the entire loop

Each stage of the loop is lit and timed during a run, and evidence passages are listed in the evidence panel with their similarity score.

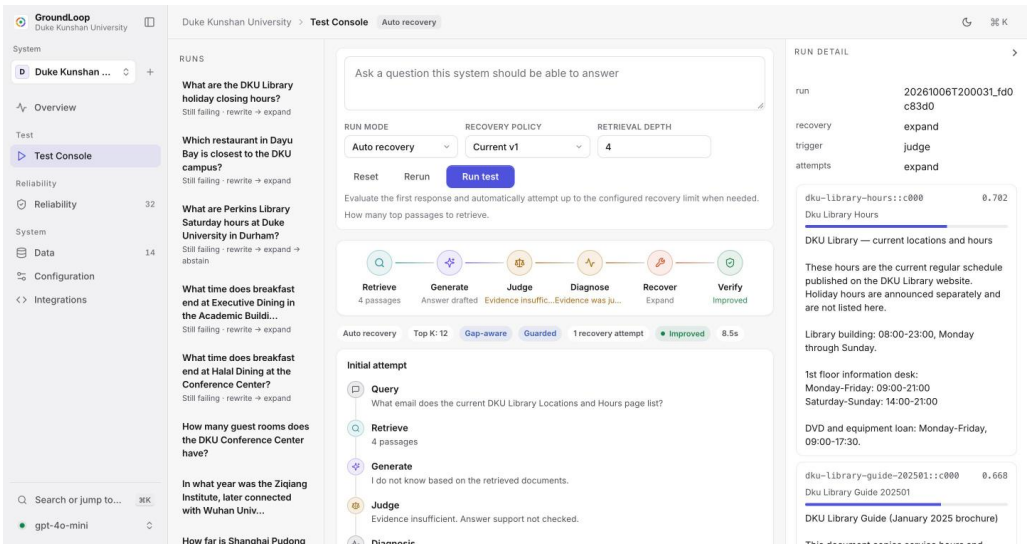



Figure 2.2: The Test Console after an automatic recovery. The loop strip shows the stage results, and the right panel lists the retrieved passages with their scores.

Each run is saved and can be reviewed later on the Reliability page; Figure 2.3 shows an example. The first answer is "I do not know" because the required passage could not be retrieved. The judge found the evidence insufficient, and so the loop expanded retrieval from 4 to 12 passages. The answer now shows the email address of the library, with a citation to the passage it used. The verification box states what the verdict means and does not mean.

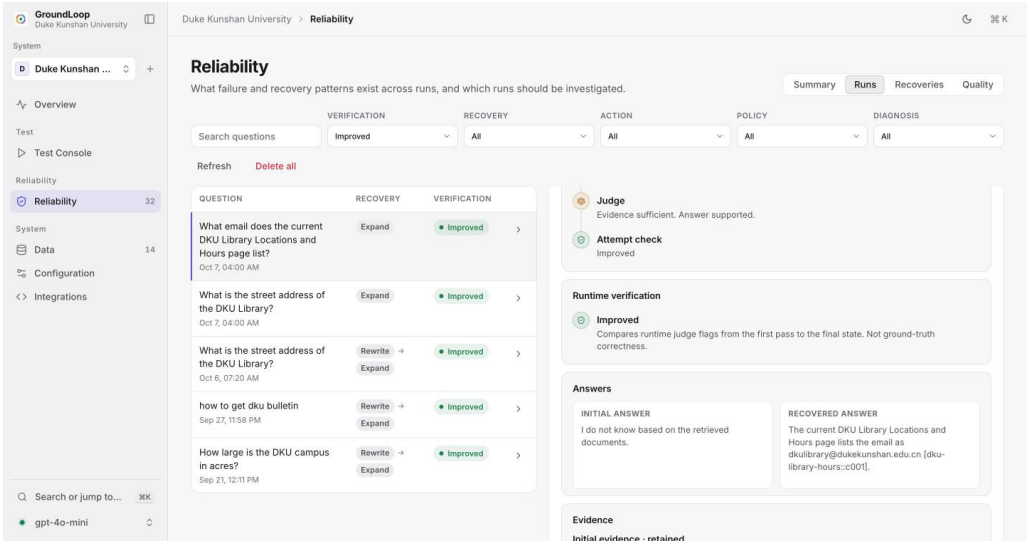



Figure 2.3: A recovered run in the Reliability view, with the runtime verdict and the initial and recovered answers side by side.

The Reliability summary in Figure 2.4 tallies the number of times recovery was needed across all runs of a system, the number of diagnoses and actions that took place, the distribution of verdicts and the judge's flags before and after the first pass and final answer. Runs can be filtered by verdict, action, policy and diagnosis; this is how the cases mentioned in Chapter 4 were identified.

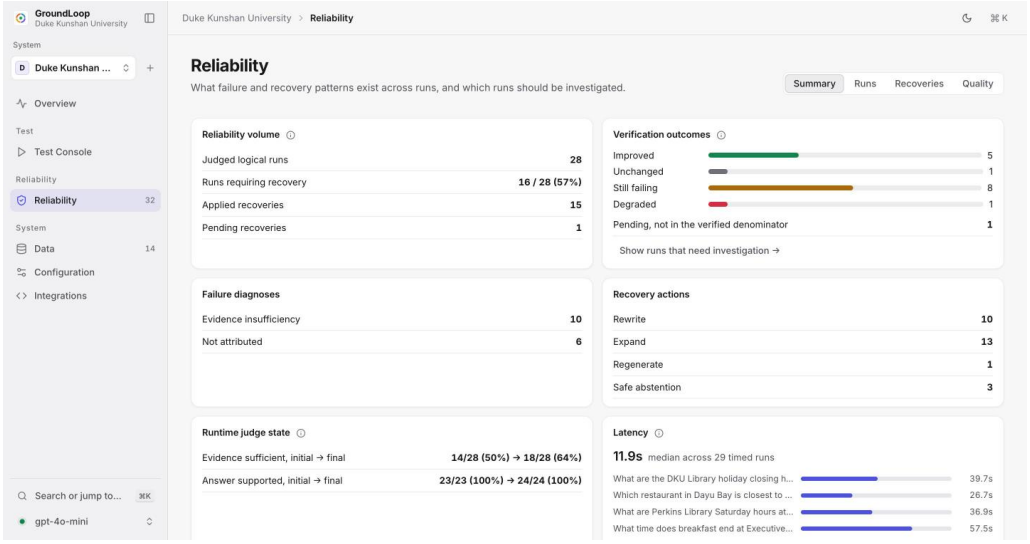



Figure 2.4: The Reliability summary for the Duke Kunshan system.

An authenticated external API allows an external RAG application to submit its own question, answer and passages and obtain the judge flags, diagnosis and suggested recovery. A public demo mode fixes one system in place, does not allow any changes, and limits runs to a certain

number per day.

## 2.4 Design Decisions

Several decisions influenced the product. They are all tracked in architecture decision records in the repo and the alternatives explored are documented.

- **A plain web page instead of a framework.** Using a FastAPI adapter for static files stayed in one language and didn't require a build step. Streamlit would have mixed pipeline code with interface code, and using React would have added another toolchain.
- **In-memory retrieval.** Using cosine similarity over local files was sufficient for dozens of documents, and all the rankings were inspectable.
- **Single hop retrieval.** Diagnosis becomes unreliable beyond single-hop retrieval chains, and the metrics of the project rely on the diagnosis being checkable.
- **Bounded recovery, safe abstention, operator control.** The loop has a bounded budget of actions, doesn't repeat any actions, and abstains instead of keeping an answer if its checks are rejected. Confirm mode is provided because having a wrong automatic repair is worse than a pause.

## 2.5 Engineering and Quality

The software was developed in small pull requests reviewed thoroughly before merging, each assigned to an issue, and runs under continuous integration with Python 3.11 and 3.12 and Node 20. It contains 473 automated Python tests and 59 JavaScript tests, which use scripted model stand-ins and thus require no API key. I performed an end-to-end audit of the backend in October 2026. Four separate reviews and live reruns uncovered 19 issues, including one scorer that incorrectly scored correct clock ranges, one rewrite that could end up "improved" with an unsupported answer, and one request that could freeze a benchmark for ten minutes; each issue was fixed with a test that failed before the fix. Two additional issues, the planted document masquerading as a passage and the false fix in Section 4.4, now have deterministic guards in place.

# Chapter 3

# EVALUATION METHOD

This chapter describes how the loop inside GroundLoop was measured. The study runs used the same engine as the console, in automatic mode.

## 3.1 Models and Collections

`gpt-4o-mini` with temperature 0 is used for generation, judging, regeneration, and query rewriting. Passages are embedded with `text-embedding-3-small`. The same chat model is used for generator and judge. Documents are split into chunks of at most 500 characters with 80 character overlap. A query retrieves the $k$ closest chunks by cosine similarity.

Each prompt that takes retrieved text takes each chunk wrapped in a tag that denotes its id, escapes any occurrence of a tag present in a document, and declares that the chunk text is data and never part of instructions. A citation to a chunk that the model was not shown is removed from its answer. These changes were made after reviewing an answer where a chunk header that had been planted was indistinguishable from a real header.

Each collection consists of plain text documents and a gold question set. Each gold row contains a question string, an answerability label, accepted answer string(s), and supporting document strings. Questions are asked that are known to be unanswerable, so that abstentions can be scored. The three study collections are summarized in Table 3.1. The documents contain superseded

notices, near-duplicates of facilities, and facts denied in the documents.

| Collection | Domain | Source | Files | Answerable | Not answerable |
|---|---|---|---|---|---|
| Harbor Institute | Research campus | Fictional | 11 | 23 | 9 |
| Duke Kunshan University | University campus | Public facts | 14 | 30 | 10 |
| Kestrel Aero Components | Aerospace parts | Fictional | 16 | 12 | 4 |

Table 3.1: Study collections and gold question counts.

I signed off each gold row on 11 October 2026. An audit of the scorer and gold set on 15 and 16 October resulted in four changes, each with reason recorded. Harbor `seed-007` ("which shuttle stop is closest to the boat museum?") was relabeled to answerable because the shuttle document states that Library Plaza is the stop to walk to; Harbor has 23 answerable and 9 unanswerable questions. An answer that correctly rejects the false premise of an unanswerable question ("the Annex has no group rooms") now counts as an abstention, using accepted rejection phrases if they are listed for the question. An answer that states a near-neighbor's fact counts as an attempt even if it hedges. Clock ranges are compared as two times. Common refusal wordings count as abstentions. These changes were made after a large number of answers had already been generated. The earlier single-depth runs, rescored under these changes, are available in Table A.1 of Appendix A.

## 3.2 Evaluation, Recovery, and Verification

The grounding judge takes question, the retrieved chunks, and the answer, and outputs two signals indicating whether the evidence would answer the question, and whether each fact mentioned in the answer appears in the evidence. The model does not have access to outside knowledge, does not receive fluency support, and never sees gold. These two checks can override the supported label:

- **Span check:** The model extracts clock times, emails, ordinals, specification tokens, and numbers of three or more digits. If any of these spans are missing in the retrieved text, the answer is rejected.
- **Source check:** If a question is of the form "what does [source] say about...", and the answer cites a different source without mentioning what the source being asked says, then

the answer is not supported.

A computed retrieval signal marks a hit as weak if the top score is less than 0.15 or $(s_1 - s_2)/s_1$ is less than 0.02; these values were chosen by hand.

An answer marked unsupported on the first pass and with enough evidence becomes *unsupported generation*. Insufficient evidence is only assigned after a second retrieval, becoming a *retrieval miss* if evidence would have been sufficient or *evidence insufficiency* if not. This is the default recovery policy. The study budget is two recoveries, and later action never repeats an earlier one. Any final expand or regenerate that still fails a check or lacks evidence becomes an abstention, which does not count as an action from the budget, nor does an expand that cannot further expand retrieval.

| State | Action | Effect |
|---|---|---|
| Unsupported generation | Regenerate | New answer from the same chunks. |
| Evidence insufficient (first pass) | Rewrite | New query; answer again only if the new evidence is sufficient. |
| Judge accepts, retrieval signal weak | Expand | Same query, wider $k$ (1 or 2 $\rightarrow$ 8, 4 $\rightarrow$ 12, cap 20). |
| Evidence insufficient after rewrite | Abstain | "I do not know based on the retrieved documents." |
| No failure | Accept | Answer kept. |

Table 3.2: Default recovery policy.

The verifier compares the same and new flags, and returns *improved*, *degraded*, *failing*, or *unchanged*. Offline scoring, which the running system does not see, reports contains-answer for answerable questions and abstention for unanswerable questions, following SQuAD 2.0 in treating an abstention for an unanswerable question as correct [21]. It also predicts a gold failure class for the baseline's answer to each question, *no failure* if it was correct, *evidence insufficiency* if the question is unanswerable, *retrieval miss* if no document was retrieved that supported the question, and *unsupported generation* if one was retrieved but the answer was wrong or abstained. Exact match is near zero because answers are sentences and gold spans are short, so contains-answer is the quality metric.

Agreement with gold is measured in two ways:

- **F1 sign.** The runtime agrees with the gold when it has the same direction as F1.
- **Rank rule.** Each answer is either correct, abstained on an answerable question, or wrong. The runtime agrees with the gold when it has the same transition in ranks. Paraphrases within a rank do not count as a transition.

The rank rule considers an abstention continued on an unanswerable question to be *unchanged* (it was correct both times). The runtime considers it still a failure (the evidence is still missing). This cell is separately annotated.

## 3.3 Study Design

Each full gold set for a collection is run once per repeat under seven conditions, in order: the retrieve-once baseline and the loop at $k = 1$, 2, and 4, and a plain baseline at $k = 8$. Lower $k$ causes the first pass to miss more often, providing something for recovery to fix. The $k = 8$ baseline tests whether simply fetching more is better than the loop at lower cost, since the loop's own expand step fetches 8 or 12. Each set of conditions is run three times on the same code (repository commit `b125064`), and the tables show mean and range. Each run's conditions are consecutive, and latencies are comparable within each collection. Cost is an estimate from list price ($0.15 and $0.60 per million input and output tokens), not an invoice. The judge audit is the same as the previous study: 40 judge calls were sampled with a fixed seed, 20 of the 20 first-pass calls that triggered on recovery and set the support flag, and 20 accepted final calls; the labeler only saw the question, evidence, and answer, and the same two flags; the labels were reviewed after a previous AI labeling pass, and are not an independent blind rating.

# Chapter 4

# RESULTS

All numbers come from committed result folders. The sweep is in `experiments/results/20261015-v5-*` and is summarized by `scripts/summarize_sweep.py` in `experiments/rescored/v5-sweep-20261015/`.

Each figure is the mean of three repeats; no significance test is applied. A provider timeout removed `dku-020` from one Duke Kunshan run.

## 4.1 Answer Quality Across Retrieval Depth

Table 4.1 shows the ratio of answerable questions with a gold span in the answer, and the ratio of unanswerable questions on which the system abstained or correctly rejected the premise.

| | | Harbor | | Duke Kunshan | | Kestrel | |
|---|---|---|---|---|---|---|---|
| Condition | $k$ | Contains | Abstains | Contains | Abstains | Contains | Abstains |
| Baseline | 1 | 0.870 | 0.778 | 0.809 | 1.000 | 0.917 | 1.000 |
| Loop | 1 | 1.000 | 0.741 | 0.966 | 0.967 | 1.000 | 1.000 |
| Baseline | 2 | 0.913 | 0.667 | 0.933 | 0.967 | 1.000 | 1.000 |
| Loop | 2 | 0.986 | 0.741 | 1.000 | 1.000 | 1.000 | 1.000 |
| Baseline | 4 | 1.000 | 0.778 | 0.966 | 0.967 | 1.000 | 1.000 |
| Loop | 4 | 1.000 | 0.889 | 1.000 | 0.933 | 1.000 | 1.000 |
| Baseline | 8 | 1.000 | 0.852 | 1.000 | 1.000 | 1.000 | 1.000 |

Table 4.1: Contains-answer rate on answerable questions, and abstention rate on unanswerable questions. Mean of three repeats. Harbor has 23 answerable and 9 unanswerable questions, Duke Kunshan 30 and 10, Kestrel 12 and 4.

At $k = 1$ the loop increased the contains-answer rate on all three corpora: Harbor from 0.870 to 1.000, Duke Kunshan from 0.809 to 0.966, Kestrel from 0.917 to 1.000. At $k = 2$ the increases were smaller and at $k = 4$ the baseline was already at or near 1.000. The plain baseline reached 1.000 on all three corpora at $k = 8$. Abstention changed by at most one question in either direction, except on Harbor at $k = 4$, where the loop abstained on 8 of 9 unanswerable questions compared to the baseline's 7.

Contains-answer for a condition varied by at most one question between repeats.

| Collection | | $k = 1$ | $k = 2$ | $k = 4$ |
|---|---|---|---|---|
| Harbor | fixed | 3, 3, 3 | 3, 3, 3 | 1, 1, 1 |
| | broke | 0, 1, 0 | 1, 1, 0 | 0, 0, 0 |
| Duke Kunshan | fixed | 5, 5, 4 | 3, 2, 2 | 1, 1, 2 |
| | broke | 0, 1, 0 | 0, 0, 0 | 1, 0, 1 |
| Kestrel | fixed | 1, 1, 1 | 0, 0, 0 | 0, 0, 0 |
| | broke | 0, 0, 0 | 0, 0, 0 | 0, 0, 0 |

Table 4.2: Questions fixed and broken by the loop compared to the baseline (same depth).

Fixed and broke counts over 27 loop runs (three depths, three collections, three repeats) are loop 49, broke 6. I've read all six. Five are unanswerable questions that the loop correctly rejected without the rejection list containing the rejection's words (e.g. "breakfast is not served there" for `dku-032`). One is a genuine regression: Harbor `seed-007` $k = 2$, the loop abstained while the baseline was correct.

The gold decisions made after seeing answers do not drive this pattern. Scored with the gold set as it stood on 11 October, contains-answer at $k = 1$ still rises from 0.864 to 1.000 on Harbor and from 0.809 to 0.966 on Duke Kunshan, and the $k = 8$ baseline still reaches 1.000 on both. Only the abstention rates fall, because correct premise rejections are not credited.

## 4.2 Runtime Diagnosis

Table 4.3 shows the comparison between the loop's diagnosis at $k = 1$ and the gold failure class pooled across nine runs.

The loop left 163 of 165 correct answers unchanged, labeled 62 of 69 unanswerable questions as evidence insufficiency, and correctly labeled 14 of 17 retrieval misses. The loop never made an

| Gold class | Questions | Loop's diagnosis |
|---|---|---|
| No failure | 165 | left alone 163, retrieval miss 2 |
| Evidence insufficiency | 69 | evidence insufficiency 62, none 7 |
| Retrieval miss | 17 | retrieval miss 14, none 3 |
| Unsupported generation | 12 | retrieval miss 12 |

Table 4.3: Loop diagnosis vs. the gold failure class. $k = 1$, nine runs.

unsupported generation diagnosis. All 12 such cases are four questions (`dku-012`, `dku-013`, `gold-015`, `gold-024`) and three repeats. In each repeat the baseline abstained although a supporting document had been retrieved, the loop diagnosed it as a retrieval miss, expanded retrieval and answered correctly. When $k = 1$, the gold rule counts a document as found if not the passage with the answer, so these disagreements can be said to arise as much from the gold rule as from the loop.

Because the action follows from the diagnosis (Table 3.2), action suitability shows in Table 4.2.

## 4.3 Cost

Table 4.4 shows chat calls, estimated cost, and median latency per run for each collection.

| | | Harbor | | | Duke Kunshan | | | Kestrel | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Condition | $k$ | Calls | USD | s | Calls | USD | s | Calls | USD | s |
| Baseline | 1 | 32 | 0.0018 | 4.6 | 40 | 0.0022 | 5.0 | 16 | 0.0007 | 4.8 |
| Loop | 1 | 107 | 0.0127 | 10.9 | 140 | 0.0170 | 13.0 | 52 | 0.0051 | 11.4 |
| Baseline | 2 | 32 | 0.0025 | 4.9 | 40 | 0.0031 | 4.6 | 16 | 0.0009 | 5.5 |
| Loop | 2 | 111 | 0.0153 | 10.9 | 129 | 0.0178 | 9.3 | 48 | 0.0051 | 10.3 |
| Baseline | 4 | 32 | 0.0036 | 4.8 | 40 | 0.0045 | 4.3 | 16 | 0.0013 | 5.5 |
| Loop | 4 | 107 | 0.0190 | 9.3 | 129 | 0.0235 | 10.2 | 48 | 0.0064 | 11.0 |
| Baseline | 8 | 32 | 0.0060 | 4.6 | 40 | 0.0073 | 3.7 | 16 | 0.0020 | 4.9 |

Table 4.4: Chat calls per run, estimated cost per run (in US dollars), and median latency per question (in seconds). Mean across three repeats.

The loop made 3.0-3.5x as many chat calls as the same-depth baseline and took 1.9-2.6x as long to answer each question. Since the expansion strategy also passes more passages to the model, it cost 4.9-7.7x as much as the same-depth baseline. The $k = 8$ baseline cost 39-47 percent of the loop at $k = 1$ and matched or exceeded its contains-answer rate on every collection. The full sweep (9 runs across 7 conditions) cost roughly $0.47.

## 4.4 Runtime Verification

Table 4.5 compares each runtime verdict with the gold rank transition, pooled over repeats and depths.

| | Answerable | | Unanswerable | |
|---|---|---|---|---|
| Collection | Verified | Agrees | Verified | Agrees |
| Harbor | 29 | 21 | 71 | 1 |
| Duke Kunshan | 41 | 35 | 90 | 0 |
| Kestrel | 3 | 3 | 36 | 0 |
| Total | 73 | 59 | 197 | 1 |

Table 4.5: Runtime verdict against the gold rank transition, all loop runs.

On answerable questions the verdict matched gold on 59 of 73 recovered runs. On unanswerable ones it matched 1 of 197, but almost all are a kept abstention that the runtime calls *still failing* and the rank rule *unchanged*: a difference in vocabulary, not a wrong decision.

One case was a wrong answer. How long did the 1948 gale last according to the Coastal Memory Group, Harbor `gold-029` asks; that group holds nothing after 1940. The answer included a 19 hours figure from another archive, which the source check rejected. In one repeat the 19 hours was attributed to the Coastal Memory Group, which the judge accepted and the verifier judged an improvement. After the sweep a further check was added to make such figures appear in the relevant passages in the source cited; out of 1,164 answers stored it triggers only in this answer.

The 40 judge calls were audited separately. For the sufficiency flags the judge agreed with the human ratings on 39 answers; for the support flags on 39. The ratings are not independent blind ratings; these were two cases where the answer used a figure from another archive (the judge agreed) and a correct “the collection does not say” answer (the judge said unsupported).

Chapter 5

# DISCUSSION

## 5.1 When Recovery Helps

Sweep distinguishes two cases neither of which could be meaningfully analyzed in the previous study. When the first query return is sparse ($k = 1$ or 2), the loop corrected almost everything the baseline could not; 49 questions repaired against 1 real regression, the same across all three corpora, and independent of the repeats. When the first query return contains the answer ($k = 4$), there was little to correct, and the loop's impact was reduced to one question per run. The finding that the loop "barely moves accuracy" in the previous study was due to the particular conditions under which it was tested. The corrections derived mostly from expansion: the correction the loop is designed to make here is a retrieval failure, which explains the role of the 8 baseline.

## 5.2 Against a Simpler Alternative

Retrieving eight passages from the start reaches the loop's best contain-answer rate on each collection at under half the cost of the loop (in tokens) at $k = 1$ and under half the time. On these collections, the wider baseline configuration matches the quality of the loop at lower cost. What the loop buys over the narrower baseline that the wider baseline does not is a record of a diagnosis, an action, and a verdict for every question it looked at. On larger collections, where eight passages is more expensive or long context hurts the generator [22], the balance could flip;

these runs are not indicative of that.

## 5.3 Verification and the Judge

For answerable questions, the verdict's runtime agreed with the gold's rank transition in 59 out of 73 runs recovered. This is a stronger test than the previous F1-sign figure since the sweep actually made repairs that the verdict could see. The unanswerable cell is just a naming issue and should be reported separately.

The relabeling of `gold-029` is the most egregious issue in this project. The loop responded to a rejection by changing the source it named, the judge accepted the outcome, and the verifier reported a repair. This is the risk of loops which can game their own feedback to satisfy it without answering the question [15], [16]. The deterministic check closes the case. This is just a single case, and the check was added after witnessing it.

## 5.4 Relation to Prior Work and Limits

Task accuracies are reported after action in corrective and adaptive RAG systems [4], [5], [6], [7]. Here recovery helped more when retrieval was starved, and the wider search matched the improvement in accuracy with more retrieved. Adaptive-RAG advocates targeted retrieval effort only on questions that needed it [7]. This project explores the balance of the version of that trade on small datasets and finds the wider, unchanging retrieval to be cheaper. Like Joren et al., sufficiency drove abstention [12] - the scorer required phrases to reject premises, and still missed five. Limitations of design and data constrain the reading of this paper. The model generates and scores, verification reads two flags, the span and source checks apply to a few fact types, and the thresholds for retrieval were hand-tuned. Small, hand-crafted datasets were used and single-hop was assumed. The judge sample is 40 calls and not independently annotated. The need for recovery was annotated by one reviewer. There have been no other users of the console. The gold relabel, the rejection phrases, and the source checks were informed by the answers. Three repeats use one model at temperature 0.

## Chapter 6

# CONCLUSIONS

Evaluation-guided recovery fixed most of the questions that a skinny first-stage retrieval missed on three petite single-hop collections. At $k = 1$, it improved 'contains-answer' from 0.81 to 0.97 on Duke Kunshan and to 1.00 on Harbor and Kestrel (one actual regression out of 27 loop runs). It made almost no repairs when the initial retrieval was sufficient. In all experiments it made about three times as many chat calls as the same-depth baseline (taking about twice as long). A plain baseline that retrieved eight passages matched the loop's quality at less than half the cost.

The loop's self-assessment was more accurate than could be shown in the previous study: it diagnosed most failures as the gold did (76 out of 98), and its verdict matched the gold verdict in 59 out of 73 answerable recoveries. It made one confident false repair (the answer kept a wrong number, changing only the source cited). The program reveals the diagnosis, action, and verdict called for by the ChatDKU gap question. On these collections, they explain what the loop did. A recovery verdict is evidence that can be inspected, not proof that a repair was made, and a better wider retrieval is what any claim for the loop has to beat.

## 6.1 Future Work

I want to make GroundLoop into a product that other teams can use and pay for. The most straightforward way to do this is as a layer on top of an existing RAG setup: developers continue to own their own retriever and model, and sends each question, answer, and passages to our

evaluation API, and receives a diagnosis, recovery, verdict, and each run is traced and can be viewed by the developer. Three things that this project didn't have that I need to get here are testing with external users to shape the console and API not built by developers, evaluation on larger question sets written by others, with more than one model and including multi-hop questions, and an ablation to show whether diagnosis is necessary, or just broadening retrieval when the judge complains. The $k = 8$ result is as much a product question as a research question: GroundLoop needs to justify its cost in explanations, not just its accuracy.

# REFERENCES


[1] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020. arXiv: 2005.11401.

[2] Yunfan Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2023. arXiv: 2312.10997.

[3] Aditi Singh et al. *Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG*. 2025. arXiv: 2501.09136.

[4] Zhengbao Jiang et al. “Active Retrieval Augmented Generation”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023. DOI: 10.18653/v1/2023.emnlp-main.495.

[5] Akari Asai et al. “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *International Conference on Learning Representations (ICLR)*. 2024. arXiv: 2310.11511.

[6] Shi-Qi Yan et al. *Corrective Retrieval Augmented Generation*. 2024. arXiv: 2401.15884.

[7] Soyeong Jeong et al. “Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity”. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 2024. DOI: 10.18653/v1/2024.naacl-long.389.

[8] Shahul Es et al. “RAGAs: Automated Evaluation of Retrieval Augmented Generation”. In: *Proceedings of the 18th Conference of the European Chapter of the Association for*

*Computational Linguistics: System Demonstrations*. 2024, pp. 150-158. DOI: 10.18653/v1/2024.eacl-demo.16.

[9] Jon Saad-Falcon et al. "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems". In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 2024, pp. 338-354. DOI: 10.18653/v1/2024.naacl-long.20.

[10] Dongyu Ru et al. "RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation". In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. Vol. 37. 2024. arXiv: 2408.08067.

[11] Scott Barnett et al. "Seven Failure Points When Engineering a Retrieval Augmented Generation System". In: *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI (CAIN)*. 2024. DOI: 10.1145/3644815.3644945.

[12] Hailey Joren et al. "Sufficient Context: A New Lens on Retrieval Augmented Generation Systems". In: *International Conference on Learning Representations (ICLR)*. 2025. arXiv: 2411.06037.

[13] Lianmin Zheng et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. Vol. 36. 2023. arXiv: 2306.05685.

[14] Peiyi Wang et al. "Large Language Models are not Fair Evaluators". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2024, pp. 9440-9450. arXiv: 2305.17926.

[15] Jie Huang et al. "Large Language Models Cannot Self-Correct Reasoning Yet". In: *International Conference on Learning Representations (ICLR)*. 2024. arXiv: 2310.01798.

[16] Ryo Kamoi et al. "When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs". In: *Transactions of the Association for Computational Linguistics* 12 (2024), pp. 1417-1440. DOI: 10.1162/tacl_a_00713.

[17] Noah Shinn et al. "Reflexion: Language Agents with Verbal Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. arXiv: 2303.11366.

[18] Mert Cemri et al. “Why Do Multi-Agent LLM Systems Fail?” In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. 2025. arXiv: 2503.13657.

[19] Kunlun Zhu et al. *Where LLM Agents Fail and How They Can Learn From Failures*. 2025. arXiv: 2509.25370.

[20] Bingbing Wen et al. “Know Your Limits: A Survey of Abstention in Large Language Models”. In: *Transactions of the Association for Computational Linguistics* 13 (2025), pp. 529-556. DOI: 10.1162/tacl_a_00754.

[21] Pranav Rajpurkar, Robin Jia, and Percy Liang. “Know What You Don’t Know: Unanswerable Questions for SQuAD”. In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 2018, pp. 784-789.

[22] Nelson F. Liu et al. “Lost in the Middle: How Language Models Use Long Contexts”. In: *Transactions of the Association for Computational Linguistics* 12 (2024), pp. 157-173.

# SIGNATURE WORK NARRATIVE

When I came to Duke Kunshan University, I knew that I wanted to work in technology and be a software engineer. I am an Applied Math and Computational Science major with a track in Computer Science. As I went through DKU, I realized my interests became more specific. Technology was also rapidly changing during my time at DKU, especially with the rise of artificial intelligence and large language models. As I learned more through classes, research, internships, and personal projects, AI is the type of technology I am the most interested in.

The primary motivation for my Signature Work came from my research experience at DKU. I had my first taste of AI research with Professor Paul Weng. I did a few different tasks, but I eventually focused more on evaluation. This was where I started thinking about AI research where not only do you have to build the AI system, but you also need to evaluate if the AI system is good or not.

After doing research with Professor Weng over the summer, I started to work with Professor Bing Luo's research group on ChatDKU, a DKU-focused retrieval-augmented generation, or RAG, system. They took me in, and my main role was to work on the evaluation pipeline and evaluate the RAG system.

While doing this, I always had this one question in mind: "And then what?" Because with an evaluation, I would get a score telling me how good or bad the system was, but I would always think, "And then what?" With the score, I would know something was wrong or right, but I wouldn't know why it happened or what part of the pipeline was faulty or what comes next.

I also attempted to work on automated evaluation while I was with ChatDKU, but I never really

finished it because automated evaluation was more complicated than I thought. These questions then led me to my Signature Work. I started to become interested in what more could evaluation do? I wanted to know if evaluation could not only score systems, but it could also diagnose failures and be recoverable or even automated recovery. These questions led me to my Signature Work.

Three classes that really helped lay the foundation to be able to work on this project were MATH 202 - Linear Algebra, CS 101 - Introduction to Computer Science, and CS 201 - Programming and Data Structures. Although I did not take these classes thinking that I would later build an AI reliability system, they helped in different ways.

MATH 202 - Linear Algebra especially became more important to me when I started working with RAG systems. One concept that stood out to me was cosine similarity. Since in RAG systems, you can represent information with embeddings or vectors, you can use cosine similarity to find the similarity between different vectors. While working on evaluation early on, I used cosine similarity a lot when comparing different embeddings and looking at the retrieval performance. Before RAG, vectors and vector similarity was mainly mathematical to me, but through research, I started to realize how these mathematical concepts support modern AI systems. Linear algebra helped me understand more about what was going on inside the retrieval process, rather than looking at the system as a black box.

CS 101 - Introduction to Computer Science helped my Signature Work in a different way. This was my first computer science class, and this class sparked my interest in programming and software engineering. It is not so much the topics, but this class made me realize that programming is something that I want to pursue, and it helped me turn my interest in technology into something that I can actually make. Obviously, GroundLoop is a lot more advanced than what I did in CS 101, but it gave me the problem solving and programming foundation to lead to projects like this.

CS 201 - Programming and Data Structures built off of CS 101 and was more similar to the type of programming I do now. It made me more comfortable with larger programs, data structures, and development environments such as Visual Studio Code. These programs required more

organization and structure in how the program was built. This became important to my Signature Work because I did not want GroundLoop to just be a research project or a bunch of scripts. I wanted to build an actual application with an interface, an evaluation pipeline, a trace, and reliability features. CS 201 helped me think about the project not only as research, but also as software.

My capstone experience helped tie all of these academic and research experiences together. Since I was already working with ChatDKU through my capstone experience, I knew that I wanted my Signature Work to come from the problems that I was having with ChatDKU. I asked Professor Bing Luo to be my mentor, and I also worked closely with Mingxi, a PhD student from Wuhan who helped me with my ChatDKU work and my capstone.

The first part of my capstone was very similar to ChatDKU. This gave me the opportunity to work with an actual RAG system and problems in the retrieval, generation, evaluation, and system design. For the second part of my capstone, I focused more on my own Signature Work. ChatDKU then became both the foundation and motivation for GroundLoop.

My project also evolved a lot while I was working on it. Originally, I thought I would use the entire ChatDKU system and data for my Signature Work. However, I quickly realized that the ChatDKU system and context was too complex for the type of question that I wanted to answer. Because with too many things changing at once, it becomes difficult to tell why a recovery or evaluation decision was successful or not. I then decided that I would first work with a more controlled RAG environment so that I can more easily inspect the system.

My literature review also impacted how I viewed my project. Originally, I was interested in the idea of an evaluator detecting a bad answer and triggering recovery automatically. However, I discovered that evaluation guided recovery loops already exist in current research. This made me think about this question in a more interesting way: if an evaluator can trigger recovery, how much can we trust the evaluator?

Instead of taking the evaluator's verdict as gospel, I then became more focused on the reliability of the evaluation and recovery process itself. I started to become interested in questions like was the evaluator accurate in detecting the failure, was the failure recoverable, and does recovery

improve the final answer?

I also chose to build GroundLoop as software that people can use rather than just doing a research project. A lot of work in this space is still just in research papers. I saw an opportunity to take these ideas and apply them in a way that developers can use. Rather than trying to compete with existing observability platforms like LangSmith, I became interested in GroundLoop as more of a additional layer of reliability for existing RAG systems.

My project also had results that did not confirm some of my hypotheses. However, this became one of the most valuable parts of my project. In some experiments, recovery helped when retrieval was bad, but just getting more information retrieved can sometimes provide the same improvement at a lower complexity. Earlier on, I would have thought this was a disappointing result because it is not only showing that my system works, but I became more comfortable with the idea that a good result can also mean a simpler solution works just as well. Evaluation should not be about proving that my system works. It is about understanding what the system is doing.

I also used AI coding tools like Claude while developing GroundLoop. My role was still to decide what to build, how to build it, evaluate results, find problems, and make all the key decision making in the project. In some ways, using AI coding tools made evaluation even more relevant to me. Because with software becoming easier to generate, knowing how to test and evaluate the software that is produced is more important than ever.

My Signature Work has also helped me narrow down what I want to do post-DKU. I had a few research experiences at DKU, and I also had internship and software engineering experience. I am thankful for my research experience because it helped me learn how to ask better questions, test hypotheses, and be more careful with what I conclude. It also helped me realize that I prefer the applied side of technology over research itself.

During my internship experience in Shanghai and while building my own software projects, I enjoyed the process of creating things that people could use. Post-DKU, I plan to continue to work as a software engineer and building applications that use AI. I am also interested in pursuing a master's degree, but I am more interested in applied programs that involve computer science, AI, software engineering, and product development rather than a research-focused

program.

I also do not necessarily see GroundLoop ending when I submit my Signature Work. One of my goals is to continue developing it as a product and releasing it. I want to see if it becomes something that developers or companies would use and pay for. Regardless of whether or not it becomes a successful company, this project is what I want my career to be: software engineering, AI, evaluation, and product development.

Looking back on my Signature Work, many parts of my experience at DKU led me to where I am now. CS 101 made me want to continue programming. CS 201 made me more comfortable building software. Linear algebra later helped me understand some of the mathematical foundations behind the AI systems that I was evaluating. My research with Professor Paul Weng introduced me to AI evaluation. My work with Professor Bing Luo, Mingxi, and ChatDKU gave me an actual system to work with and see its limitations. My capstone then gave me the opportunity to take those limitations and form my own question and ultimately GroundLoop. Everything came full circle with this experience, showing me that not only do you have to build an AI system, but you also need to understand when it works, why it does not, and what comes next.

# Appendix A

# ADDITIONAL MATERIAL

## A.1 Software Availability

GroundLoop's source code, tests, benchmark configurations, and every result folder cited in this paper are kept in a private repository. The author will grant access to examiners on request.

A read-only snapshot of the GroundLoop console, with recorded Duke Kunshan runs, can be opened in any browser at https://sieguerra.github.io/groundloop/ (source: https://github.com/sieguerra/groundloop). It shows the interface and stored traces but does not run new questions.

## A.2 Study Configurations

The depth sweep used `experiments/configs/v5_{harbor,dku,kestrel}_depth_sweep.json` at repository commit `b125064`. Its snapshots are `experiments/results/20261015-v5-<collection>-r<1,2,3>`, and the summary is `experiments/rescored/v5-sweep-20261015/`. The earlier $k = 4$ study is `20261010T203121` (Harbor), `20261010T203115` (Duke Kunshan), and `20261010T202849` (Kestrel), rescored in `experiments/rescored/scoring-v4-gold-2026-10-16/`. Judge-audit files are in `experiments/judge-audit/20261011/`. The trace schema is in

`docs/trace-schema.md`.

## A.3 Earlier Runs, Rescored

| | Contains-answer | | Unanswerable abstention | |
|---|---|---|---|---|
| Folder | Stored | Rescored | Stored | Rescored |
| Harbor `20261010T203121` | 0.864 / 0.864 | 1.000 / 1.000 | 0.400 / 0.500 | 0.667 / 0.778 |
| Duke Kunshan `20261010T203115` | 0.767 / 0.800 | 0.967 / 1.000 | 0.800 / 0.900 | 1.000 / 1.000 |
| Kestrel `20261010T202849` | 0.917 / 0.917 | 1.000 / 1.000 | 1.000 / 1.000 | 1.000 / 1.000 |

Table A.1: Earlier $k = 4$ runs, baseline / loop, as stored and as rescored (`experiments/rescored/scoring-v4-gold-2026-10-16`).

## A.4 Use of AI Tools

Generative AI tools assisted with this project. They helped write and review software and run the benchmarks. AI was also used for grammar and paraphrasing. The research question, design decisions, claims, interpretation, and conclusions are mine.

## A.5 Product Tour

Two further screens complement Figures 2.2 to 2.4. The Overview (Figure A.1) is the landing page of a system: it counts how many runs passed through each stage of the loop and lists the runs that still need attention. The Configuration page (Figure A.2) shows a system's pipeline and settings and the four steps for building an editable system on one's own documents.

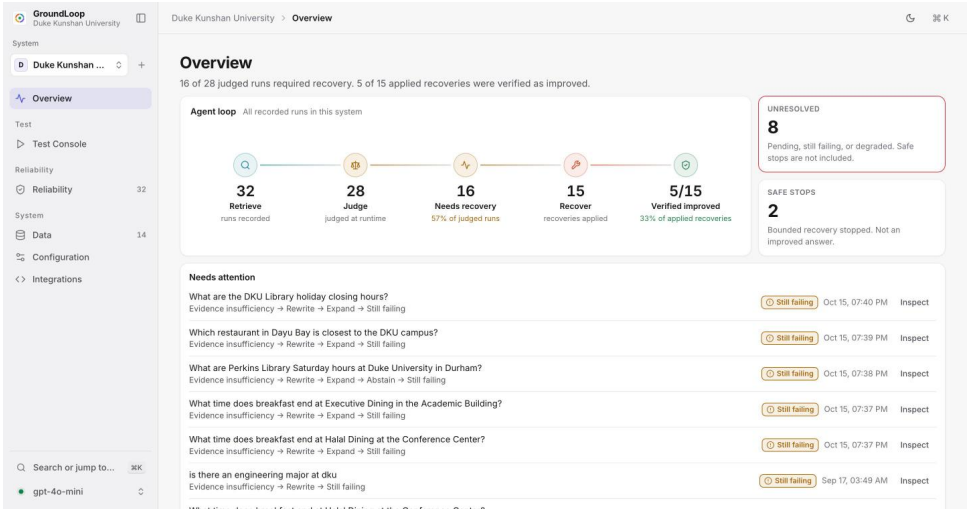



Figure A.1: The Overview page for the Duke Kunshan system.

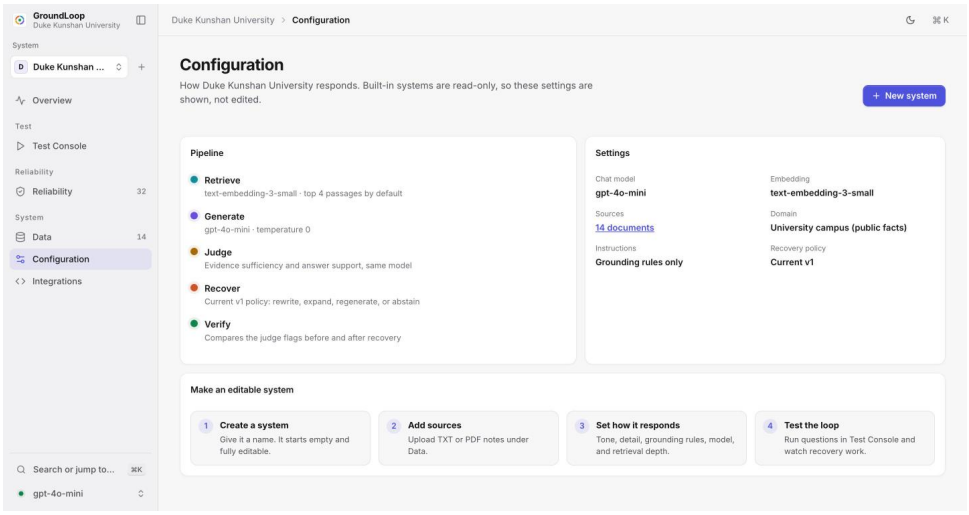



Figure A.2: The Configuration page. Built-in systems are read-only; a new system is fully editable.

Three more screens show the rest of the console. The Runs view of the Reliability page (Figure A.3) lists every run with its recovery path and verdict; opening a run shows its loop and a timeline of each stage. The Data page (Figure A.4) lists a system's source documents and shows the text of each one. The Integrations page (Figure A.5) documents the external evaluation API with examples ready to copy.

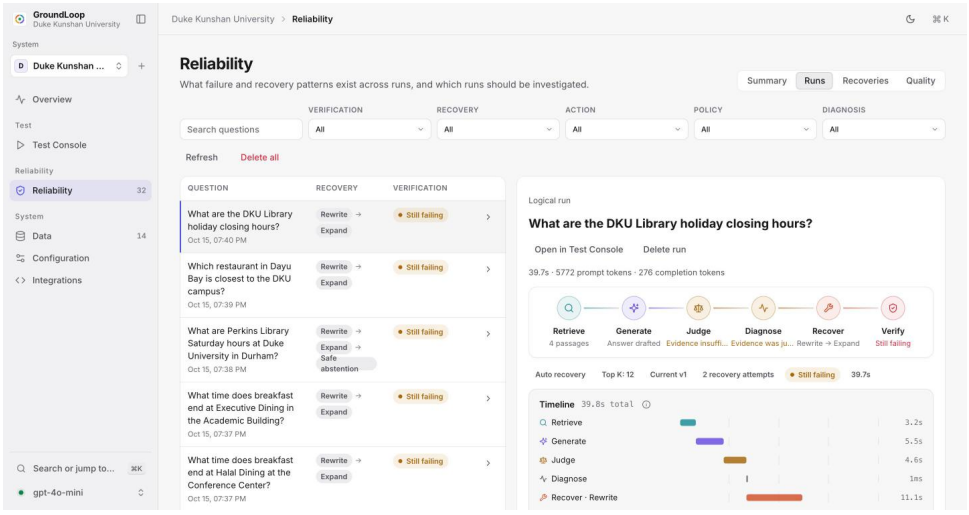



Figure A.3: The Runs view of the Reliability page, with one run opened.

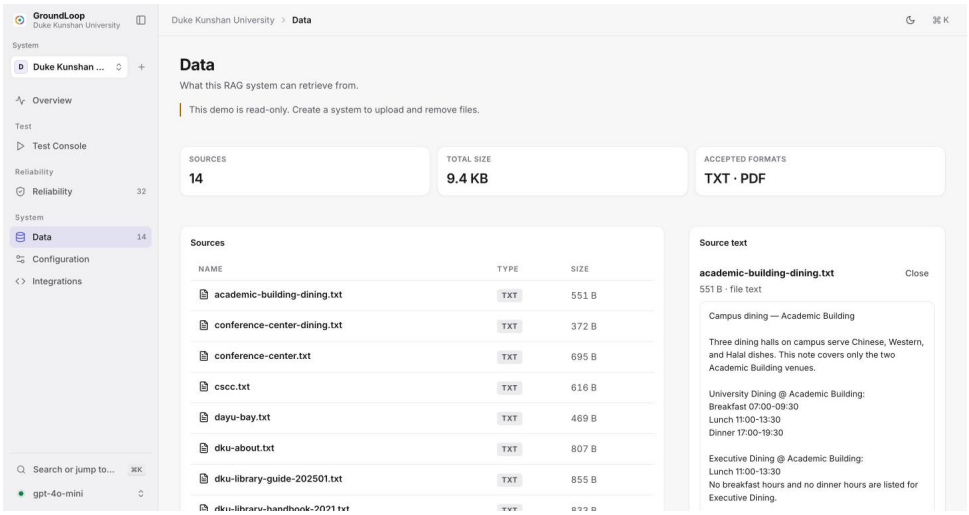



Figure A.4: The Data page, listing a system's sources with one open.

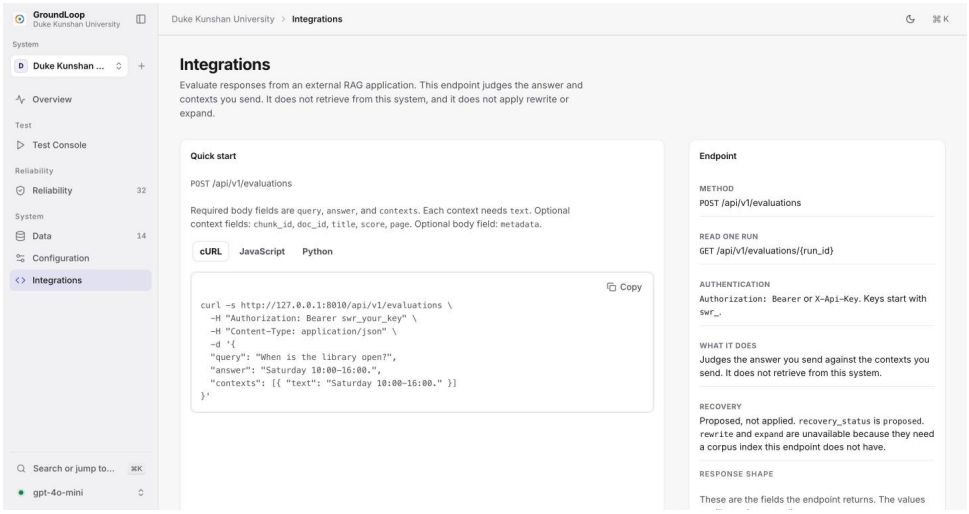



Figure A.5: The Integrations page for the external evaluation API.